WEBRUM-02: SPA Instrumentation

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 2 of 10 | **Created:** March 2026 | **Last Updated:** 10/23/2026

Overview

Single-Page Applications (SPAs) present unique challenges for Real User Monitoring. Unlike traditional multi-page apps where each navigation triggers a full page load, SPAs dynamically update content via JavaScript — meaning route changes, data fetches, and user interactions happen without the browser firing standard navigation events. Dynatrace addresses this with automatic SPA detection, XHR/fetch monitoring, route change tracking, and custom action naming.

Table of Contents

1. SPA Monitoring Challenges
2. RUM Injection Methods
3. Route Change Detection
4. Framework-Specific Instrumentation
5. Custom Action Naming
6. XHR and Fetch Monitoring
7. Validating SPA Instrumentation
8. Hybrid Apps: Mobile WebView + Desktop
9. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	At least one SPA with RUM injection active
Permissions	`storage:events:read`, `storage:entities:read`
Knowledge	Familiarity with SPA frameworks (Angular, React, or Vue.js)
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. SPA Monitoring Challenges

Traditional RUM relies on browser navigation events (`load`, `DOMContentLoaded`) to detect page loads. SPAs break this model:

Challenge	Traditional App	SPA
Page load detection	Browser `load` event fires on every page	Only fires once on initial load
Route changes	Full HTTP request per page	JavaScript updates the URL and DOM
Data fetching	Server renders HTML with data	Client fetches data via XHR/fetch after load
User interactions	Links trigger navigation	Clicks trigger state changes without navigation
Timing measurement	Navigation Timing API covers full load	Custom timing needed for virtual route changes

Dynatrace handles these by:

- **Monitoring `history.pushState` and `hashchange`** events for route detection
- **Tracking XHR/fetch calls** triggered by user interactions
- **Grouping related XHR calls** into a single user action
- **Detecting framework-specific lifecycle events** for Angular, React, and Vue.js

> **Important:** SPA monitoring requires the **Async Web Requests** and **SPA route changes** capture settings to be enabled in your web application configuration.

2. RUM Injection Methods

For SPAs, choose the injection method that matches your deployment architecture:

Automatic Injection (the default where OneAgent runs)

If your SPA is served by a web server with OneAgent installed in full-stack mode, injection happens automatically. OneAgent modifies the HTML response to include the RUM JavaScript snippet in the `<head>` section.

```
&lt;!-- Automatically injected by OneAgent --&gt;
&lt;script type="text/javascript" src="/ruxitagentjs_..." data-
dtconfig="..."&gt;&lt;/script&gt;
```

Manual Injection

For SPAs served from CDNs or static hosting (S3, Netlify, Vercel) where no server-side OneAgent exists:

```
&lt;!-- Add to &lt;head&gt; before other scripts --&gt;
&lt;script type="text/javascript" src="https://your-
activegate/jstag/managed/.../ruxitagentjs_..."
        data-dtconfig="..."&gt;&lt;/script&gt;
```

Agentless RUM (JavaScript Tag)

For environments where you cannot install OneAgent at all:

1. Navigate to **Digital Experience > Web Applications > Your App > Setup**
2. Select **Agentless monitoring**
3. Copy the generated `<script>` tag
4. Paste into your application's `index.html`

> **Tip:** For SPAs built with modern bundlers (Webpack, Vite), you can inject the RUM tag via an HTML plugin to keep it out of source code.

3. Route Change Detection

Dynatrace detects SPA route changes by monitoring:

- **`history.pushState()`** / **`history.replaceState()`** — Used by most modern SPAs
- **`hashchange`** **events** — Used by hash-based routing (e.g., `/#/page`)
- **Framework-specific routing** — Angular Router, React Router, Vue Router

In New RUM, each detected route change is a soft navigation: a navigation event, and a user action with `user_action.type == "soft_navigation"`. The fields to query:

Field	Description
`user_action.type`	`soft_navigation` — *"triggered by a soft navigation that led to a view change"* (Classic: `RouteChange`)
`page.detected_name`	The page the route change happened on
`characteristics.has_navigation`	`true` on the navigation event the route change emits
`duration`	Duration of the user action

> **Sources:** User actions — semantic dictionary (DT docs) — *"The user action was triggered by a soft navigation that led to a view change."*

Let's query route change actions to validate detection:

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type: "Load" -&gt; "hard_navigation", "RouteChange" -&gt;
"soft_navigation",
//                     "Xhr" -&gt; "same_view", "Custom" -&gt; "api"
(documented values, corrected 10/05/2026)
//   connection.type         -&gt; no web RUM equivalent
(network.connection.type is OneAgent for Mobile
//                              only; network.protocol.name is the OSI
protocol, e.g. "http")
// THE EVENT CHARACTERISTIC MATTERS AS MUCH AS THE FIELD: navigation-timing
fields
// (performance.dom_interactive, performance.load_event_end) live on
navigation events
// (characteristics.has_navigation) and are unpopulated on page summaries
// (characteristics.has_page_summary), so a page-summary filter silently
empties them.
// ttfb.* is the opposite — it lives on page summaries. Select with the stable
has_* flags, not
// characteristics.classifier (corrected 09/28/2026): the docs call classifier
"not intended for
// query usage" and Semantic Dictionary 1.349 removes it from the user-event
models (re-listed in SD 1.350, still not for query use).
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.has_page_summary
== true
//                                        (was characteristics.classifier —
"not intended for query usage", SD 1.349)
//   action.type                       -&gt; user_action.type
(hard_navigation | soft_navigation | same_view | api)
//   action.name                       -&gt; page.detected_name
//   LCP / CLS                         -&gt;
web_vitals.largest_contentful_paint / web_vitals.cumulative_layout_shift
//                                        (stable — the fields Dynatrace's own
DQL examples use), falling back to
//                                        the experimental lcp.start_time /
cls.value (corrected 10/14/2026)
//   app.name                          -&gt; frontend.name (stable; an array
```

```
on user.sessions)
// UNITS DIFFER BETWEEN THE TWO FAMILIES. web_vitals.largest_contentful_paint
is a DURATION, so
// `/ 1ms` turns it into milliseconds; lcp.start_time is a PLAIN NUMBER
already in milliseconds,
// and dividing it by 1ms yields null. The CWV cells therefore read
// coalesce(web_vitals.largest_contentful_paint / 1ms, lcp.start_time) and
// coalesce(web_vitals.cumulative_layout_shift, cls.value), against the
2500/4000 ms and 0.1/0.25 thresholds.
// Why both: on the validation tenant every user event comes from the
synthetic engine, which fills
// lcp.* / cls.* but not web_vitals.* (0 of 243,494 page summaries in 30 days,
10/14/2026). That
// is a property of synthetic-monitor data, not a sign that web_vitals.* is
rarely populated.
// Route change actions in the last hour — verify SPA detection is working.
// Route changes are "soft_navigation" (corrected 10/05/2026); "same_view" is
an interaction
// followed by an XHR or fetch request, the New RUM form of a classic Xhr
action.
fetch user.events, from:-1h
| filter characteristics.has_user_action == true
| filter user_action.type == "soft_navigation"
| summarize {route_count = count(), avg_duration = avg(duration)}, by:
{page.detected_name, frontend.name}
| sort route_count desc
| limit 20
```

4. Framework-Specific Instrumentation

Angular

Angular applications use the Angular Router with `history.pushState`. Dynatrace automatically detects:

- Route changes via `NavigationEnd` events
- Zone.js async tracking for XHR/fetch calls
- Component lifecycle events

Configuration tips:
- Enable **Async Web Requests** capture for Zone.js tracked calls
- Set **User action naming rules** to map Angular routes to meaningful names
- For lazy-loaded modules, ensure the RUM agent loads before the router initializes

React

React applications typically use React Router or Next.js routing:

- Route changes detected via `history.pushState` (React Router v5+) or `startTransition` (React 18+)
- Component renders tracked through XHR/fetch activity
- No Zone.js — relies on standard browser API monitoring

Configuration tips:
- For Next.js SSR/SSG, use automatic injection on the Node.js server
- For client-only React apps, use manual or agentless injection
- Set **User action naming** to use URL path segments for clearer action names

Vue.js

Vue applications use Vue Router with `history.pushState` or hash mode:

- Route changes detected via `router.afterEach` hook monitoring
- Vuex/Pinia store actions tracked through associated XHR calls

Configuration tips:
- For Nuxt.js, inject the RUM tag in `nuxt.config.js` head configuration
- Turn on **Capture fetch() requests** under **Capturing > Async web requests and SPAs > Generic support** — check it rather than assume it is on (see section 6)

> **Note:** Regardless of framework, the key setting is enabling **SPA route changes** in your web application configuration. Without it, Dynatrace only captures the initial page load.

```
// Compare action types across applications — identify SPA vs traditional apps
fetch user.events, from:-24h
| filter characteristics.has_user_action == true
| summarize action_count = count(), by:{frontend.name, user_action.type}
| sort frontend.name asc, action_count desc
```

If an SPA shows only `hard_navigation` actions and no `soft_navigation` or `same_view` actions, SPA instrumentation may not be configured correctly. A traditional multi-page application legitimately shows mostly `hard_navigation`. The classic values `Load`,

`RouteChange` and `Xhr` never appear on New RUM data, so a check that looks for them reports a working setup as broken.

5. Custom Action Naming

Default action names like "loading of page /app/dashboard" or "route change to /app/users/12345" can be noisy — especially with dynamic URL segments (IDs, tokens). Custom action naming rules let you normalize names:

Naming Rule Configuration

Configure via **Settings > Web and mobile monitoring > User action naming**:

Rule Type	Example Pattern	Result
URL cleanup	`/users/{id}` → `/users/{*}`	Groups all user detail pages
CSS selector	Use `data-dtname` attribute	`<button data-dtname="Checkout">`
JavaScript variable	Read from SPA router state	Dynamic names from app context
Regex replacement	`/api/v[0-9]+/` → `/api/`	Normalize API versions

RUM API for Custom Actions

For fine-grained control, use the Dynatrace JavaScript RUM API:

```
// Start a custom action
const actionId = dtrum.enterAction('Checkout - Payment Step');

// ... perform user interaction ...

// End the action
dtrum.leaveAction(actionId);
```

The `dtrum` calls above are the RUM Classic JavaScript API. In the New RUM model, a user action reported through the Dynatrace API carries `user_action.type == "api"` —

"The user action was reported via the Dynatrace API" — and `characteristics.is_api_reported == true`. Filter on those rather than on the classic `Custom` action type, which never matches New RUM data.

> **Sources:** User actions — semantic dictionary (DT docs) — *"The user action was reported via the Dynatrace API."*

6. XHR and Fetch Monitoring

SPAs rely heavily on asynchronous API calls. Dynatrace monitors both `XMLHttpRequest` and the Fetch API to:

- **Correlate API calls with user actions** — Which button click triggered which API call?
- **Measure API response times** — How long did the backend take?
- **Detect failed requests** — HTTP 4xx/5xx responses and network errors (`characteristics.has_failed_request`)
- **Track third-party calls** — Calls to external APIs, CDNs, analytics services

Enabling Fetch API Monitoring

Out of the box, RUM Classic captures page loads; XHR actions are something you activate. Under **Generic support**, `fetch()` and `XmlHttpRequest` are **separate toggles**, so a SPA whose calls go through `fetch()` needs its own switch turned on:

Application settings → **Capturing > Async web requests and SPAs** → **Generic support** → **Capture fetch() requests** (and **Capture XmlHttpRequest (XHR)** for XHR-based calls)

> **Sources:** Configure Real User Monitoring Classic to capture XHR actions (DT docs) — *"When you activate XHR-action support, you add visibility into each kind of user interaction, not just the regular page loads that are captured by default."*

```
// Unit trap (08/12/2026): New RUM timing fields such as lcp.start_time and
ttfb.value
// are PLAIN NUMBERS already in milliseconds, not durations. `field / 1ms`
yields null on them —
// silently, so a chart of nulls looks like "no data". Compare and aggregate
them directly.
// XHR action performance — identify slow API calls impacting user experience.
// has_request also matches the document request of every page navigation and
every
// failed-request error event; excluding navigations leaves the XHR/fetch
calls.
fetch user.events, from:-1h
| filter characteristics.has_request == true
| filter not(coalesce(characteristics.has_navigation, false))   // null, not
false, on non-navigations
| summarize {xhr_count = count(),
    avg_duration = avg(duration),
    p95_duration = percentile(duration, 95)},
    by:{page.detected_name}
| sort p95_duration desc
| limit 15
```

```
// XHR error rate by action — find failing API calls. error.count is a SESSION
field and is
// null on every user event, so it cannot count failed requests (corrected
10/05/2026).
fetch user.events, from:-24h
| filter characteristics.has_request == true
| summarize {total = count(),
    errors = countIf(characteristics.has_failed_request == true)},
    by:{page.detected_name}
| fieldsAdd error_rate_pct = round(toDouble(errors) / toDouble(total) * 100.0,
decimals: 2)
| filter total &gt; 10
| sort error_rate_pct desc
| limit 10
```

7. Validating SPA Instrumentation

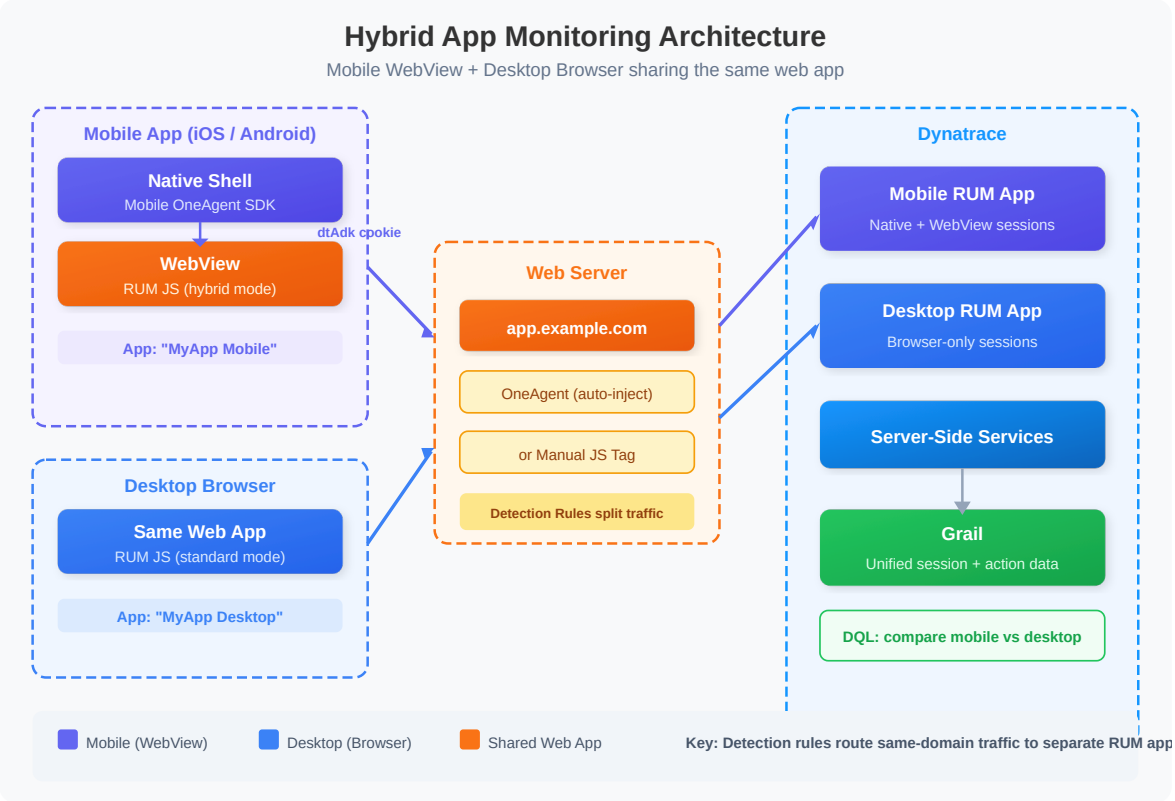
After configuring SPA monitoring, validate that instrumentation is working correctly with this checklist:

Check	DQL Validation	Expected Result
Route changes detected	Query for `user_action.type == "soft_navigation"`	Actions appear for each route
XHR/fetch captured	Query for `user_action.type == "same_view"` and `characteristics.has_request` events	API calls visible per action
Action names meaningful	Review the `page.detected_name` distribution	No generic/unclear names
No duplicate actions	Check for overlapping actions	One action per interaction
API-reported actions working	Query for `user_action.type == "api"`	Developer-reported actions appear

```
// Instrumentation health check — action type distribution per app
fetch user.events, from:-24h
| filter characteristics.has_user_action == true
| summarize total_actions = count(), by:{frontend.name, user_action.type}
| sort frontend.name asc, total_actions desc
```

8. Hybrid Apps: Mobile WebView + Desktop

A common pattern is a web application served to both desktop browsers and mobile app WebViews — the same `app.example.com` pages, but consumed in two very different contexts. Dynatrace can monitor both as separate RUM applications with full session correlation.



Architecture Overview

Client	Instrumentation	Dynatrace Application
iOS / Android app	Mobile OneAgent SDK + `hybridWebView` enabled	Mobile RUM App ("MyApp Mobile")
Desktop browser	RUM JavaScript (auto-inject or manual tag)	Web RUM App ("MyApp Desktop")

Both clients load the **same web pages** from the same origin, but Dynatrace routes them to **separate applications** using application detection rules.

Step 1: Create Two RUM Applications

In Dynatrace, create two separate applications:

Application	Type	Purpose
MyApp Desktop	Web Application	Desktop/laptop browser traffic
MyApp Mobile	Mobile Application	iOS/Android native + WebView traffic

Step 2: Configure Application Detection Rules

Detection rules route traffic from the same domain to different applications based on request characteristics.

Navigate to **Settings > Web and mobile monitoring > Application detection** and create rules in this order (higher priority first):

Priority	Rule	Condition	Target Application
1	Mobile WebView	User-Agent header **contains** `MyAppName` (your app's custom UA string)	MyApp Mobile
2	Desktop fallback	Domain **equals** `app.example.com`	MyApp Desktop

> **Tip:** Most mobile apps append a custom identifier to the WebView User-Agent string (e.g., `MyAppName/3.2.1`). Use this as the primary differentiator. If your app doesn't, add one via WebView settings:

```
// Android: Add custom User-Agent suffix
webView.getSettings().setUserAgentString(
    webView.getSettings().getUserAgentString() + " MyAppName/3.2.1"
);
```

```
// iOS: Add custom User-Agent suffix
let config = WKWebViewConfiguration()
config.applicationNameForUserAgent = "MyAppName/3.2.1"
let webView = WKWebView(frame: .zero, configuration: config)
```

Step 3: Enable Hybrid Monitoring in the Mobile SDK

Android (Gradle Plugin)

```
// build.gradle (app module)
dynatrace {
    configurations {
        defaultConfig {
            autoStart {
                applicationId "your-application-id"
                beaconUrl "https://your-env.bf.dynatrace.com/mbeacon"
            }
            hybridWebView {
                enabled true
                domains "app.example.com"   // Only instrument your own domain
            }
        }
    }
}
```

Then instrument the WebView instance:

```
// In your Activity or Fragment
import com.dynatrace.android.agent.Dynatrace

val webView = findViewById&lt;WebView&gt;(R.id.webView)
Dynatrace.instrumentWebView(webView)
webView.loadUrl("https://app.example.com")
```

iOS (plist or manual config)

Add to your `Info.plist`:

```
&lt;key&gt;DTXHybridApplication&lt;/key&gt;
&lt;true/&gt;
&lt;key&gt;DTXMonitoredDomains&lt;/key&gt;
&lt;array&gt;
    &lt;string&gt;app.example.com&lt;/string&gt;
&lt;/array&gt;
```

Or configure programmatically:

```
import Dynatrace

// In AppDelegate or SceneDelegate
Dynatrace.startupWithInfoPlistSettings()

// When creating a WKWebView
let webView = WKWebView(frame: view.bounds)
// The SDK automatically instruments WKWebView when DTXHybridApplication is
true
```

Step 4: Session Correlation (dtAdk Cookie)

When hybrid monitoring is enabled, the Mobile OneAgent SDK sets a `dtAdk` **cookie** on the WebView domain. This cookie links the WebView's web session to the native mobile session, creating a **unified session** that includes both native actions (taps, gestures) and web actions (page loads, XHR calls).

Correlation Mechanism	Description
`dtAdk` cookie	Set by Mobile SDK on the WebView; links web beacon to mobile session
`x-dtc` header	Optional, explicitly enabled — links **cross-origin** XHR actions to distributed traces where RUM cookies cannot reach; same-origin requests correlate via the RUM cookies
`applicationId`	Routes beacons to the correct Dynatrace application

> **Important:** Only add your own domains to `domains` / `DTXMonitoredDomains`. Never instrument third-party WebViews (e.g., OAuth login pages, payment providers) — this can break functionality and violate privacy policies.

Step 5: Validate the Setup

After deploying both applications, verify that:

1. **Desktop sessions** appear under "MyApp Desktop" with standard browser User-Agents
2. **Mobile WebView sessions** appear under "MyApp Mobile" with your custom UA string

3. **Mobile sessions show both native and web actions** in a single session timeline
4. **Distributed traces** connect WebView XHR calls to server-side services

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so
expand it before grouping)
//                  primary_tags.application exists only if your OpenPipeline
adds it (corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries;
web_vitals.time_to_first_byte where populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request
wait/redirect phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the documented values are "real_user" / "robot" / "synthetic"
(lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Compare user action metrics between mobile WebView and desktop browser apps
fetch user.events, from:-24h
// Replace the placeholders with your two frontend names (corrected
10/14/2026: dt.rum.application.id
// is an ID — "an 8-byte HEX string" for web apps — so comparing it with a
display name never matched)
| filter in(frontend.name, {"&lt;your-desktop-frontend&gt;", "&lt;your-mobile-
webview-frontend&gt;"})
| summarize
    {actions = count(),
    avg_duration = avg(duration),
    error_rate = toDouble(countIf(characteristics.has_error == true)) /
toDouble(count()) * 100},
    by:{frontend.name, user_action.type}
| sort frontend.name asc, actions desc
```

WebView Session Analysis

Identify sessions in the mobile app that include WebView activity — these are your hybrid users.

```
// Identify WebView sessions in the mobile app — sessions with both native and
web events.
// Classified by dt.rum.agent.type ("android" / "ios" = OneAgent for Mobile,
"javascript" = RUM
// JavaScript) — the classic "UserAction" / "Load" / "RouteChange" / "Xhr"
values never appear on
// New RUM data. Not execution-verified: the validation tenant has no hybrid
app.
fetch user.events, from:-24h
// Replace the placeholders with the frontend names of the app's native and
WebView parts
// (corrected 10/14/2026: dt.rum.application.id is an ID and never equals a
display name)
| filter in(frontend.name, {"&lt;your-mobile-frontend&gt;", "&lt;your-mobile-
webview-frontend&gt;"})
| summarize
    {native_events = countIf(in(dt.rum.agent.type, {"android", "ios"})),
    web_events = countIf(dt.rum.agent.type == "javascript")},
    by:{dt.rum.session.id}
| filter web_events &gt; 0
| sort web_events desc
| limit 20
```

Performance Comparison: Mobile WebView vs Desktop

WebView performance typically differs from desktop browsers due to device constraints, network conditions, and rendering engine differences.

```
// Compare page load performance: mobile WebView vs desktop browser
fetch user.events, from:-24h
| filter characteristics.has_navigation == true
// Replace the placeholders with your two frontend names (corrected
10/14/2026: dt.rum.application.id
// is an ID and never equals a display name)
| filter in(frontend.name, {"&lt;your-desktop-frontend&gt;", "&lt;your-mobile-
webview-frontend&gt;"})
| summarize
    {p50_duration = percentile(duration, 50),
    p95_duration = percentile(duration, 95),
    avg_dom_interactive = avg(performance.dom_interactive),
    actions = count()},
    by:{frontend.name}
```

9. Summary and Next Steps

In this notebook, we covered:

- **SPA monitoring challenges** — Why traditional page-load tracking fails for SPAs
- **Injection methods** — Automatic, manual, and agentless approaches
- **Route change detection** — How Dynatrace captures SPA navigation
- **Framework-specific tips** — Angular, React, and Vue.js considerations
- **Custom action naming** — Normalizing action names for cleaner analytics
- **XHR/fetch monitoring** — Tracking API call performance and errors
- **Hybrid app setup** — Monitoring mobile WebView and desktop browser as separate RUM applications

Next Steps

- **WEBRUM-03: Core Web Vitals** — Measure LCP, INP, and CLS with DQL
- **WEBRUM-05: Error Analysis** — Deep dive into JavaScript error tracking in SPAs

References

- User actions in RUM Classic (DT docs)
- Android mobile frontends (DT docs)
- Web applications in RUM Classic (DT docs)

- Link cross-origin XHR user actions and their distributed traces (DT docs)
- Customize RUM with the JavaScript API (DT docs)
- Customize RUM with the JavaScript API (DT docs)
- User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*
- Semantic Dictionary changelog 1.349 (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.